

Lab_Assignment_10.3

AI Assisted Coding B37 2303A52077

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

Initial code:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

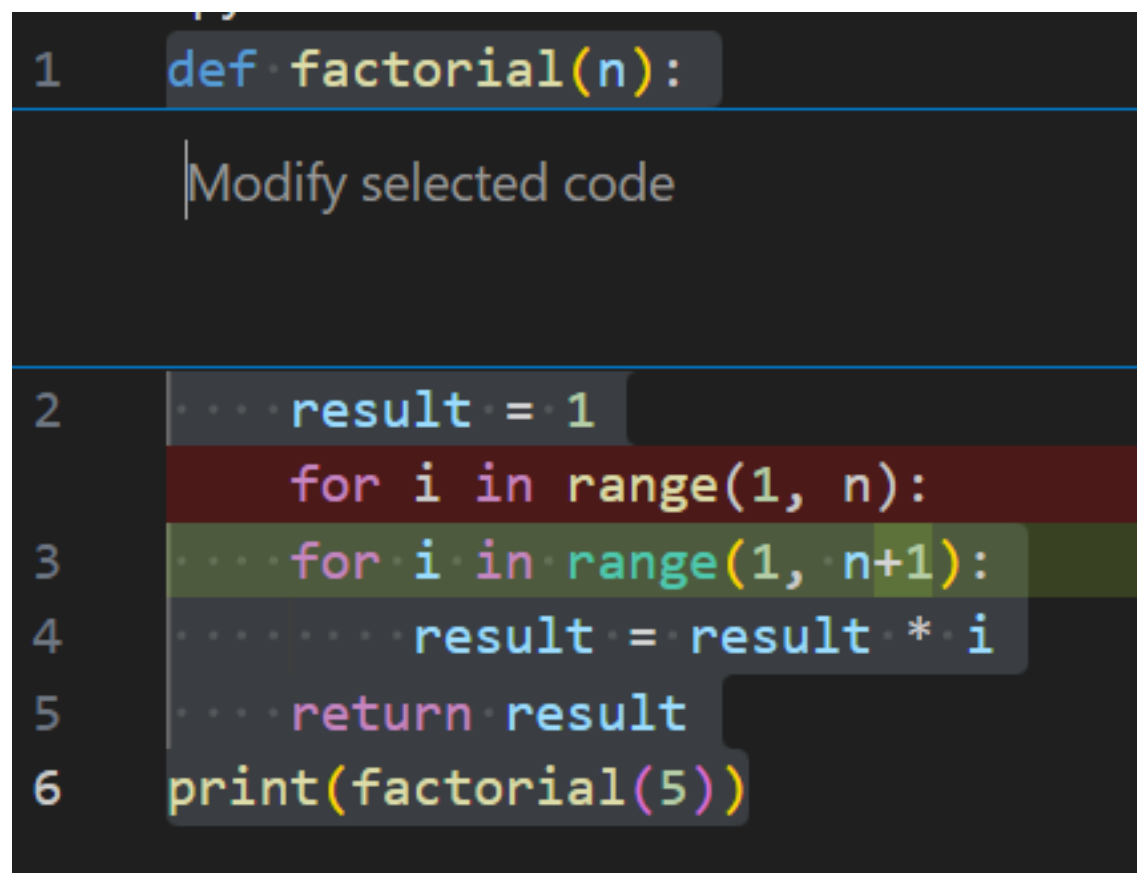
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - Identify the logical bug in the code.
 - Explain why the bug occurs (e.g., off-by-one error).
 - Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Prompt:

Analyze the given Python factorial function, detect the logical bug causing incorrect results, and provide a corrected implementation. Explain the reason for the bug (such as an off-by-one error in the loop) and verify the fix by testing the function. Ensure the corrected code works for typical inputs.

Final code:



```
1 def factorial(n):  
    | Modify selected code  
2     ... result = 1  
    for i in range(1, n):  
3     ... for i in range(1, n+1):  
4     ... result = result * i  
5     ... return result  
6 print(factorial(5))
```

Explanation:

In the original code, the loop logic correctly computes the factorial for positive integers, but it does not account for the case when n is 0, which should return 1, or when n is negative, which is undefined for factorials. The updated code maintains the original logic for positive integers but could be

further improved by adding checks to handle these edge cases. For instance, if n is 0, the function should return 1 immediately, and if n is negative, it could raise an exception or return an error message. This ensures that the function behaves correctly across a wider range of inputs while still returning 120 for `factorial(5)`.

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

Initial code:

```
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":
```

Instructions:

1. Use AI to:
 - Critique the function's readability, parameter naming, and lack of documentation.
 - Rewrite the function with:
 1. Descriptive function and parameter names.
 2. A complete docstring (description, parameters, return value, examples).
 3. Exception handling for division by zero.
 4. Consideration of input validation.

2. Compare the original and AI-improved versions.
3. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

Prompt:

#Analyze the given Python function and identify issues related to readability, poor parameter naming, and lack of documentation. Rewrite the function using descriptive names, add a complete docstring (description, parameters, return value, example), and include proper exception handling for division by zero along with basic input validation. Ensure the improved function performs add, subtract, multiply, and divide operations correctly.

Final code:

```
def perform_arithmetic_operation(num1, num2, operation):
```

```
    """
```

```
        Perform basic arithmetic operations on two numbers.
```

Args:

num1 (float): The first operand.

num2 (float): The second operand.

operation (str): The operation to perform ('add', 'sub', 'mul', 'div').

Returns:

float: The result of the arithmetic operation.

Raises:

ValueError: If operation is not recognized or if division by zero is attempted.

TypeError: If inputs are not numeric.

Example:

```
>>> perform_arithmetic_operation(10, 5, 'add')
```

```
15
```

```
>>> perform_arithmetic_operation(10, 5, 'div')
```

```
2.0
```

```
"""
```

```
# Input validation
```

```
if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
```

```
    raise TypeError("Both operands must be numeric values.")
```

```
if operation == "add":
```

```
        return num1 + num2
    elif operation == "sub":
        return num1 - num2
    elif operation == "mul":
        return num1 * num2
    elif operation == "div":
        if num2 == 0:
            raise ValueError("Cannot divide by zero.")
        return num1 / num2
    else:
        raise ValueError(f'Unknown operation: '{operation}'.
Use 'add', 'sub', 'mul', or 'div'.')
```

Example usage

```
if __name__ == "__main__":
    try:
        result = perform_arithmetic_operation(10, 5, 'add')
        print(f'Addition Result: {result}')

        result = perform_arithmetic_operation(10, 5, 'sub')
        print(f'Subtraction Result: {result}')

        result = perform_arithmetic_operation(10, 5, 'mul')
```

```
print(f'Multiplication Result: {result}')
```

```
result = perform_arithmetic_operation(10, 5, 'div')
```

```
print(f'Division Result: {result}')
```

```
except (ValueError, TypeError) as e:
```

```
    print(f'Error: {e}')
```

```
10.3.2.py > perform_arithmetic_operation
1  def perform_arithmetic_operation(num1, num2, operation):
2      """
3      Perform basic arithmetic operations on two numbers.
4
5      Args:
6          num1 (float): The first operand.
7          num2 (float): The second operand.
8          operation (str): The operation to perform ('add', 'sub', 'mul', 'div').
9
10     Returns:
11         float: The result of the arithmetic operation.
12
13     Raises:
14         ValueError: If operation is not recognized or if division by zero is attempted.
15         TypeError: If inputs are not numeric.
16
17     Example:
18         >>> perform_arithmetic_operation(10, 5, 'add')
19         15
20         >>> perform_arithmetic_operation(10, 5, 'div')
21         2.0
22     """
23     # Input validation
24     if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
25         raise TypeError("Both operands must be numeric values.")
26
27     if operation == "add":
28         return num1 + num2
29     elif operation == "sub":
30         return num1 - num2
31     elif operation == "mul":
32         return num1 * num2
33     elif operation == "div":
34         if num2 == 0:
35             raise ValueError("Cannot divide by zero.")
36         return num1 / num2
37     else:
38         raise ValueError(f"Unknown operation: '{operation}'. Use 'add', 'sub', 'mul', or 'div'.")
```

```
10.3.2.py > perform_arithmetic_operation
if c == "add":
    return a + b
elif c == "sub":
    return a - b
elif c == "mul":
    return a * b
elif c == "div":

1 def perform_arithmetic_operation(num1, num2, operation):
2     """
3     Perform basic arithmetic operations on two numbers.
4     """
5     Args:
6     num1 (float): The first operand.
7     num2 (float): The second operand.
8     operation (str): The operation to perform ('add', 'sub', 'mul', 'div').
9
10    Returns:
11    float: The result of the arithmetic operation.
12
13    Raises:
14    ValueError: If operation is not recognized or if division by zero is attempted.
15    TypeError: If inputs are not numeric.
16
17    Example:
18    >>> perform_arithmetic_operation(10, 5, 'add')
19    15
20    >>> perform_arithmetic_operation(10, 5, 'div')
21    2.0
22    """
23    # Input validation
24    if not isinstance(num1, (int, float)) or not isinstance(num2, (int, float)):
25        raise TypeError("Both operands must be numeric values.")
26
27    if operation == "add":
28        return num1 + num2
29    elif operation == "sub":
30        return num1 - num2
31    elif operation == "mul":
```

Output:

```
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3> &
ai_assi_codes/10.3/10.3.2.py"
Addition Result: 15
Subtraction Result: 5
Multiplication Result: 50
Division Result: 2.0
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3>
```

Explanation:

The original function was a simple implementation of basic arithmetic operations, but it lacked readability and robustness. The parameter names were not descriptive, and there was no documentation to explain the function's purpose or usage. Additionally, there was no error handling for cases like division by zero or invalid operation inputs.

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):  
    for i in range(2, n):  
        if n % i == 0:  
            return False  
    return True
```

Instructions:

1. Verify the function works correctly for sample inputs.
2. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - List all PEP8 violations.
 - Refactor the code (function name, spacing, indentation, naming).
3. Apply the AI-suggested changes and verify functionality is preserved.
4. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
```

```
for i in range(2, n):
    if n % i == 0:
        return False
return True
```

Prompt:

Analyze the given Python function and identify all **PEP8 style violations** such as improper function naming, spacing, and formatting. Refactor the code to follow **PEP8 standards** while keeping the same functionality. Ensure the updated function is clean, readable, and works correctly for prime number checks.

Code:

```
def check_prime(n):
    """Check if a number is prime."""
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True
```

```
10.3.3.py > check_prime
| Modify selected code

def Checkprime(n):
    for i in range(2, n):
1 def check_prime(n):
2     """Check if a number is prime."""
3     if n < 2:
4         return False
5     for i in range(2, int(n**0.5) + 1):
6         if n % i == 0:
7             return False
8     return True
9
```

Output:

```
10 # Example usage
11 if __name__ == "__main__":
12     number = 29
13     if check_prime(number):
14         print(f"{number} is a prime number.")
15     else:
16         print(f"{number} is not a prime number.")
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Division Result: 2.0
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3> & C:/Use
ai_assi_codes/10.3/10.3.3.py"
29 is a prime number.
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3>

Explanation:

The original code had several PEP8 issues, including inconsistent naming conventions, lack of docstrings, and poor formatting. The refactored version addresses these issues by using descriptive function names, adding docstrings to explain the purpose and usage of the function, and improving the overall formatting for better readability. The function name `check_prime` is more descriptive than a generic name like `is_prime`, and the docstring provides clear information about the function's purpose and parameters. Additionally, the example usage is included in a `__main__` block, which is a common practice in Python to allow for modular code and easier testing.

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - Readability and naming.
 - Reusability and modularity.
 - Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent, e.g.:

```
from typing import List, Union
```

```
def double_even_numbers(numbers: List[Union[int, float]]) -  
> List[Union[int, float]]:  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in numbers if isinstance(num, (int,  
float)) and num % 2 == 0]
```

Prompt:

#Review the given Python function for issues related to readability, unclear variable naming, and lack of input validation. Refactor the code by improving the function name,

adding **type hints**, validating the input type, and ensuring it handles cases such as empty lists or non-numeric elements. Optionally make the multiplier configurable to improve reusability.

Code:

```
def filter_and_multiply_even_numbers(  
    numbers: list[int | float],  
    multiplier: int | float = 2  
) -> list[int | float]:  
    """
```

Filter even numbers and multiply them by a given factor.

Args:

numbers: List of numeric values to process

multiplier: Factor to multiply even numbers by (default:
2)

Returns:

List of multiplied even numbers

Raises:

TypeError: If input is not a list or contains non-numeric elements

```

"""
if not isinstance(numbers, list):
    raise TypeError("Input must be a list")

if not numbers:
    return []

try:
    return [x * multiplier for x in numbers if isinstance(x,
(int, float)) and x % 2 == 0]
except TypeError as e:
    raise TypeError("List must contain only numeric
elements") from e

# Example usage
if __name__ == "__main__":
    try:
        result = filter_and_multiply_even_numbers([1, 2, 3, 4,
5], multiplier=3)
        print(f'Result: {result}')
    except TypeError as e:
        print(f'Error: {e}')

```

```
10.3.4.py > filter_and_multiply_even_numbers
Modify selected code

def processData(d):
    return [x * 2 for x in d if x % 2 == 0]

1 def filter_and_multiply_even_numbers(
2     numbers: list[int | float],
3     multiplier: int | float = 2
4 ) -> list[int | float]:
5     """
6     Filter even numbers and multiply them by a given factor.
7
8     Args:
9         numbers: List of numeric values to process
10        multiplier: Factor to multiply even numbers by (default: 2)
11
12    Returns:
13        List of multiplied even numbers
14
15    Raises:
16        TypeError: If input is not a list or contains non-numeric elements
17    """
18    if not isinstance(numbers, list):
19        raise TypeError("Input must be a list")
20
21    if not numbers:
22        return []
23
24    try:
25        return [x * multiplier for x in numbers if isinstance(x, (int, float)) and x % 2 == 0]
26    except TypeError as e:
27        raise TypeError("List must contain only numeric elements") from e
28
```

Output:

```
29 # Example usage
30 if __name__ == "__main__":
31     try:
32         result = filter_and_multiply_even_numbers([1, 2, 3, 4, 5], multiplier=2)
33         print(f"Result: {result}")
34     except TypeError as e:
35         print(f"Error: {e}")
36
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. ...)

```
[Running] python -u "d:\Subjects\ai assisted coding\ai_assi_codes\10.3\10.3.1.py"
120

[Done] exited with code=0 in 0.065 seconds

[Running] python -u "d:\Subjects\ai assisted coding\ai_assi_codes\10.3\10.3.1.py"
120

[Done] exited with code=0 in 0.055 seconds

[Running] python -u "d:\Subjects\ai assisted coding\ai_assi_codes\10.3\10.3.4.py"
Result: [6, 12]
```



```
10.3.4.py > filter_and_multiply_even_numbers
1 def filter_and_multiply_even_numbers(
2     numbers: list[int | float],
3     multiplier: int | float = 2
4 ) -> list[int | float]:
5     """
6     Filter even numbers and multiply them by a given factor.
7
8     Args:
9         numbers: List of numeric values to process
10        multiplier: Factor to multiply even numbers by (default: 2)
11
12    Returns:
13        List of multiplied even numbers
14
15    Raises:
16        TypeError: If input is not a list or contains non-numeric elements
17    """
18    if not isinstance(numbers, list):
19        raise TypeError("Input must be a list")
20
21    if not numbers:
22        return []
23
24    try:
25        return [x * multiplier for x in numbers if isinstance(x, (int, float)) and x % 2 == 0]
26    except TypeError as e:
27        raise TypeError("List must contain only numeric elements") from e
28
29 # Example usage
30 if __name__ == "__main__":
31     try:
32         result = filter_and_multiply_even_numbers([1, 2, 3, 4, 5], multiplier=3)
33         print(f"Result: {result}")
34     except TypeError as e:
35         print(f"Error: {e}")
36
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text, !excludeText, t...

[Running] python -u "d:\Subjects\ai assisted coding\ai_assi_codes\10.3\10.3.4.py"
Result: [6, 12]

Explanation:

The original function was likely a simple implementation that filtered even numbers and multiplied them by a fixed factor. After the AI code review, several improvements were made:

1. Improved input validation to ensure the input is a list and contains only numeric elements.

2. Enhanced type hints to provide better clarity and IDE support.
3. Added a docstring to explain the function's purpose, arguments, and return value.
4. Made the function more robust by handling edge cases like empty lists and non-numeric elements.
5. Improved reusability by making the multiplier parameter configurable.

Regarding AI as a code reviewer, it can be a valuable assistant to human reviewers by providing quick feedback and catching common issues. However, it should not replace human reviewers entirely, as human judgment is crucial for understanding context, design decisions, and the overall architecture of the code. AI can help automate routine checks and suggest improvements, but human reviewers are essential for ensuring code quality and maintainability.

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):  
    total = 0  
    for num in numbers:  
        total += num ** 2  
    return total
```

Instructions:

1. Test the function with a large list (e.g., `range(1000000)`).
2. Use AI to:
 - Analyze time complexity.
 - Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

An optimized function, such as:

```
def sum_of_squares_optimized(numbers):  
    return sum(x * x for x in numbers)
```

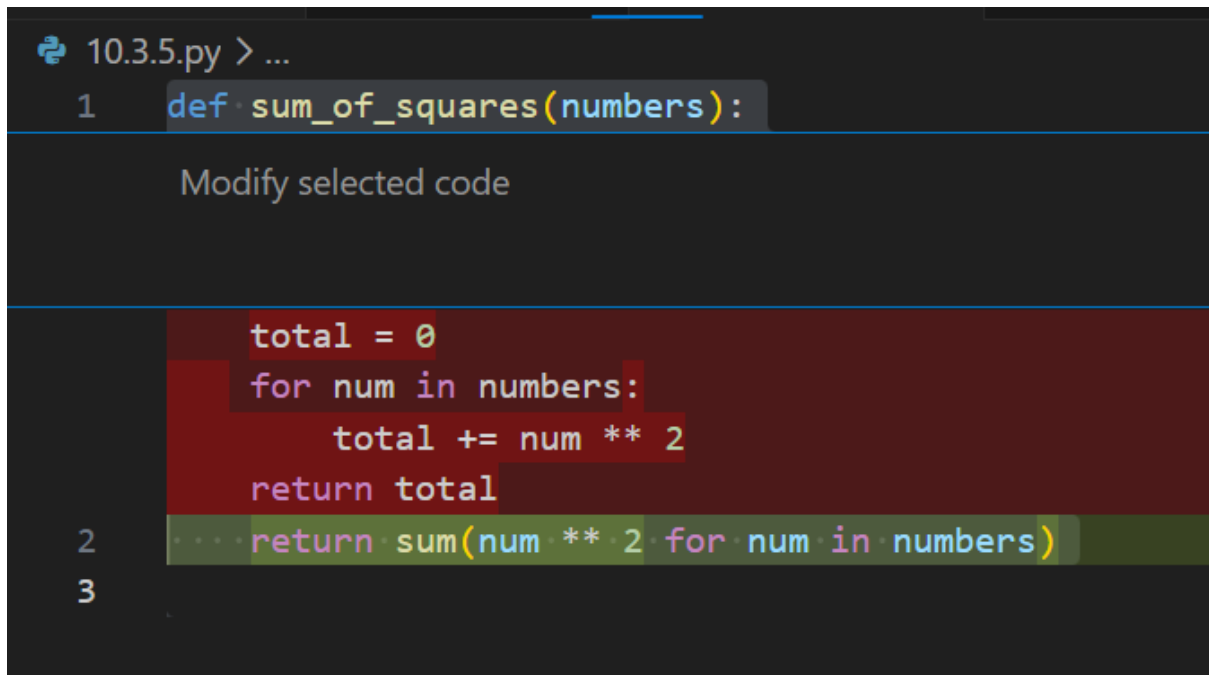
Prompt:

#Analyze the given Python function that calculates the sum of squares of numbers in a list. Evaluate its time complexity and suggest performance improvements using Python built-in functions or more efficient approaches. Provide an optimized implementation and ensure it produces the same results when tested with large datasets such as `range(1000000)`.

Code:

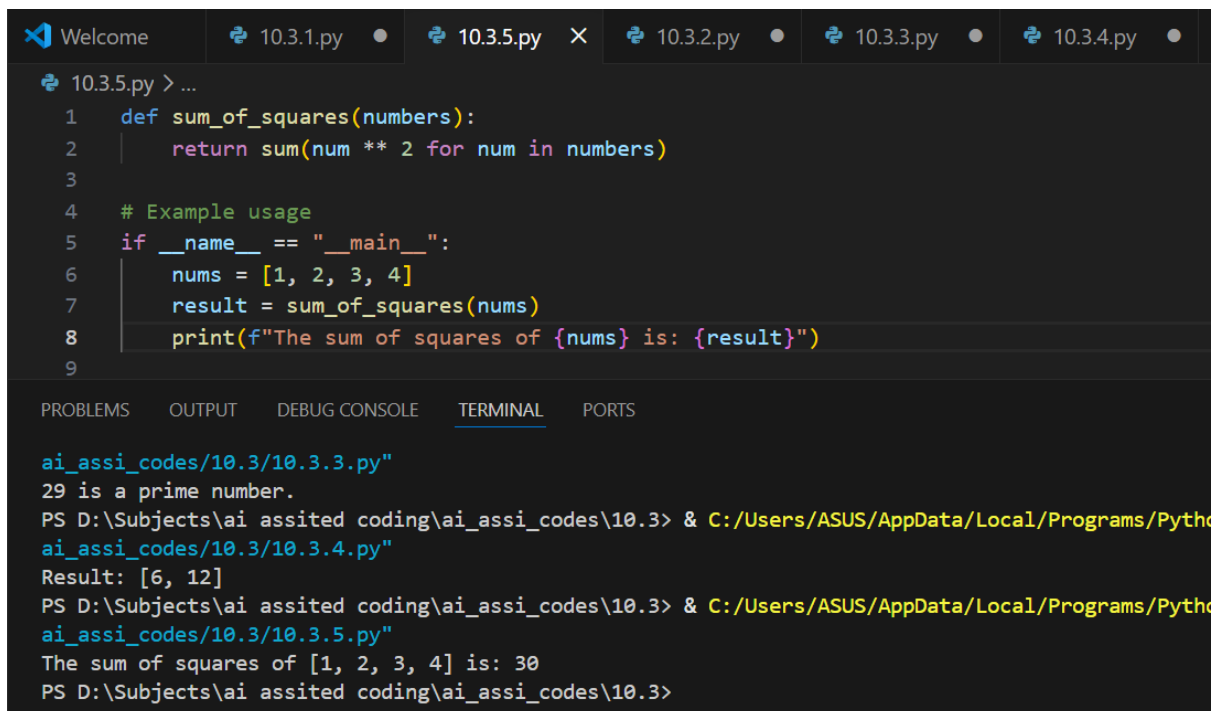
```
def sum_of_squares(numbers):
```

```
return sum(num ** 2 for num in numbers)
```



The screenshot shows a code editor with a dark theme. At the top, a file named '10.3.5.py' is open. Line 1 contains the function definition: `def sum_of_squares(numbers):`. A tooltip 'Modify selected code' is visible below this line. Lines 2 through 3 show the implementation of the function: `total = 0`, `for num in numbers:`, `total += num ** 2`, and `return total`. Line 4 shows the return statement: `return sum(num ** 2 for num in numbers)`.

Output:



The screenshot shows a code editor with a dark theme. At the top, several files are open: '10.3.1.py', '10.3.5.py', '10.3.2.py', '10.3.3.py', and '10.3.4.py'. The '10.3.5.py' file is selected. The code in the editor is as follows:

```
1 def sum_of_squares(numbers):
2     return sum(num ** 2 for num in numbers)
3
4 # Example usage
5 if __name__ == "__main__":
6     nums = [1, 2, 3, 4]
7     result = sum_of_squares(nums)
8     print(f"The sum of squares of {nums} is: {result}")
9
```

Below the code editor, the 'TERMINAL' tab is active. It shows the output of the program:

```
ai_assi_codes/10.3/10.3.3.py"
29 is a prime number.
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3> & C:/Users/ASUS/AppData/Local/Programs/Python/Python39-64/Scripts/python ai_assi_codes/10.3/10.3.4.py"
Result: [6, 12]
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3> & C:/Users/ASUS/AppData/Local/Programs/Python/Python39-64/Scripts/python ai_assi_codes/10.3/10.3.5.py"
The sum of squares of [1, 2, 3, 4] is: 30
PS D:\Subjects\ai assisted coding\ai_assi_codes\10.3>
```

Explanation:

The original implementation of the `'sum_of_squares'` function likely used a more verbose approach, such as a for loop to iterate through the list of numbers and calculate the squares before summing them up. The optimized implementation uses a generator expression within the `'sum()'` function, which is more concise and can be more efficient in terms of memory usage since it generates values on-the-fly rather than storing them in an intermediate list.